

NFFF SimNIBS Quick-Start Tutorial

From TMS FEM Simulation to Quantitative E-Field Data

NeuroEM Flux Force Five (NFFF) - EEE 488 Capstone

Purpose: give every NFFF team member the minimum SimNIBS experience needed to run a standardized TMS finite-element simulation, find and visualize the result mesh, and extract useful electric-field data in Python.

Training philosophy: This is a standardized onboarding exercise. Everyone initially uses the same head model, coil, placement, and basic settings. Individual project roles diverge after this workflow is working.

1. Install SimNIBS

Download and install SimNIBS using the official installer:

https://simnibs.github.io/simnibs/build/html/installation/simnibs_installer.html

For Windows, use the standard installer and accept the default installation locations. Typical folders may include:

C:\Users\<your_username>\SimNIBS-4.6

C:\Users\<your_username>\simnibs_env

Important: Keep NFFF project data separate from the SimNIBS program and example directories.

2. Create the NFFF Working Directory

A STRONG recommendation to Create a dedicated tutorial directory (without setting up dedicated folders, it will become a Cluster F... over time)

```
NFFF Team Tutorial\  
simulations\  
  tutorial_01\  
    extracted_data\  
      notebooks\  
        screenshots\
```

The first simulation output will go into tutorial_01. This avoids having to hunt through installation folders for large .msh result files.

[INSERT SCREENSHOT HERE]

Folder structure: simulations, extracted_data, notebooks, screenshots

3. Follow the Official SimNIBS TMS GUI Tutorial

Official GUI tutorial:

<https://simnibs.github.io/simnibs/build/html/tutorial/gui.html>

NFFF SimNIBS Quick-Start Tutorial

NFFF instruction: Follow the "Setting up a TMS Simulation" process. Do NOT perform the tDCS simulation for this exercise.

4. Configure the Standardized TMS Simulation

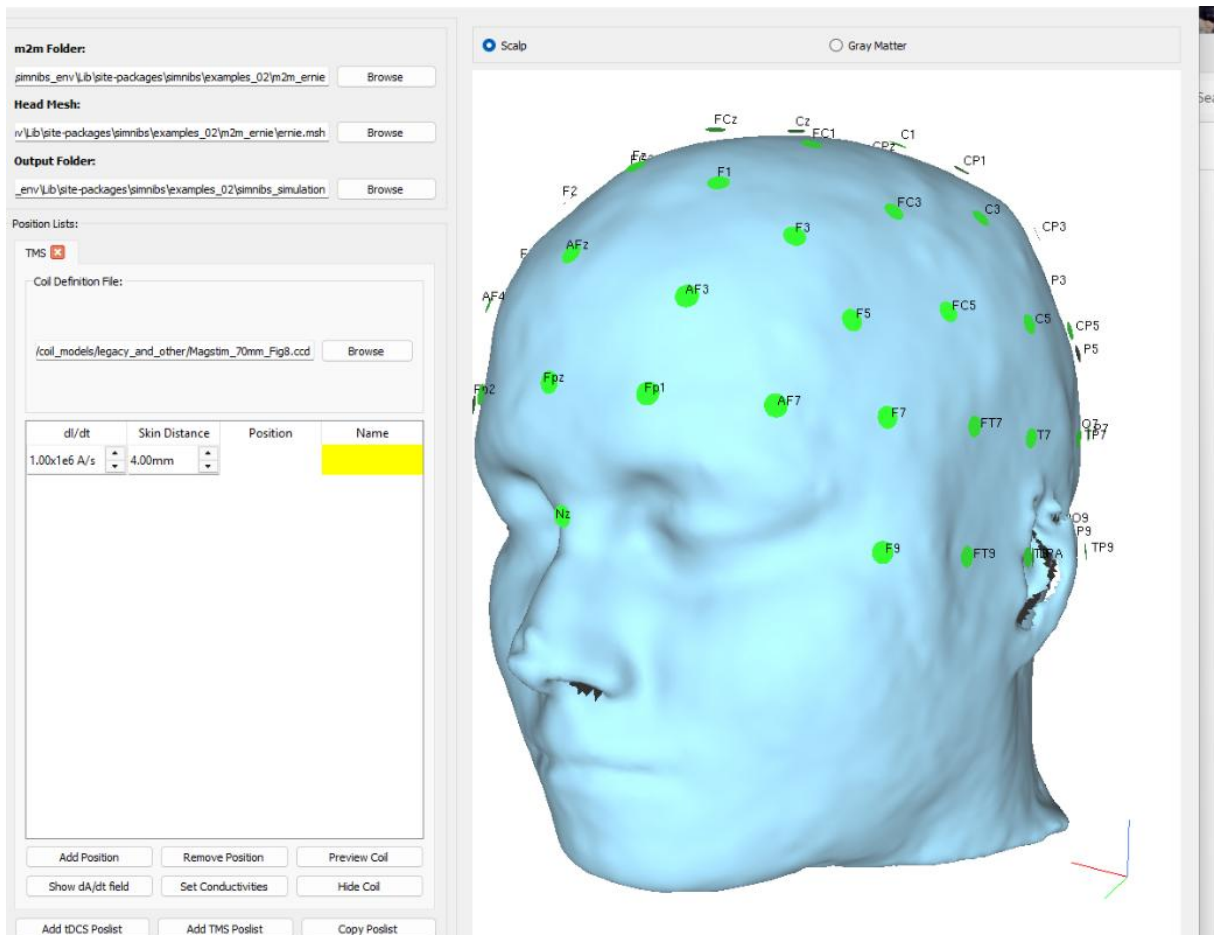
Launch the SimNIBS GUI and use the supplied **ernie** example head model. Everyone initially uses the same anatomy so results are directly comparable.

Setting	Tutorial value
Head model	ernie / m2m_ernie
Coil	Magstim_70mm_Fig8.ccd
dI/dt	1.00×10^6 A/s
Skin Distance	4.00 mm
Position	C3
Name	C3

Set the Output Folder to:

<your NFFF Team Tutorial folder>\simulations\tutorial_01

Checkpoint: Verify ernie, the Magstim 70-mm figure-8 coil, dI/dt = 1.00×10^6 A/s, skin distance = 4.00 mm, and the tutorial_01 output directory.



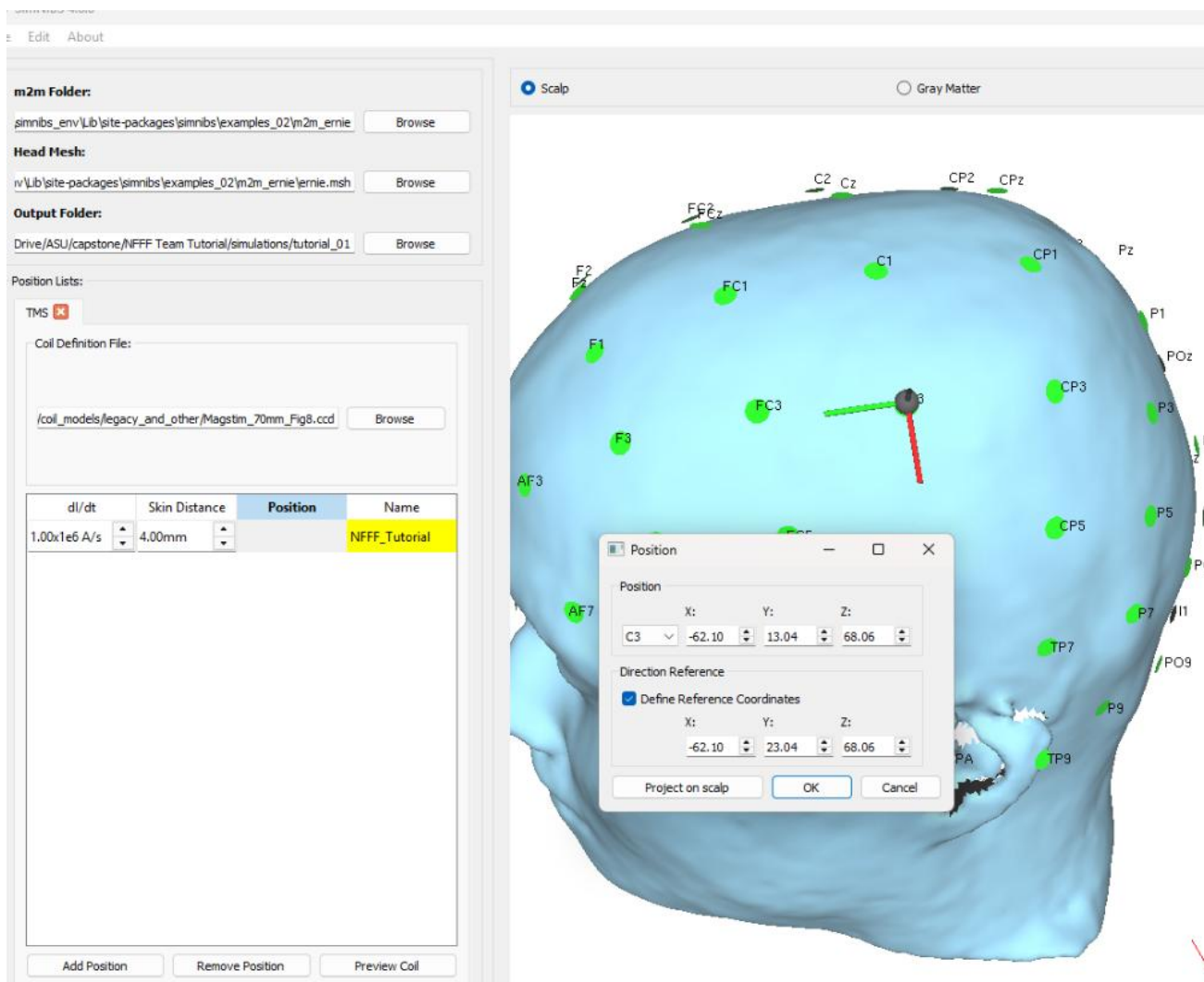
NFFF SimNIBS Quick-Start Tutorial

5. Position the Coil at C3

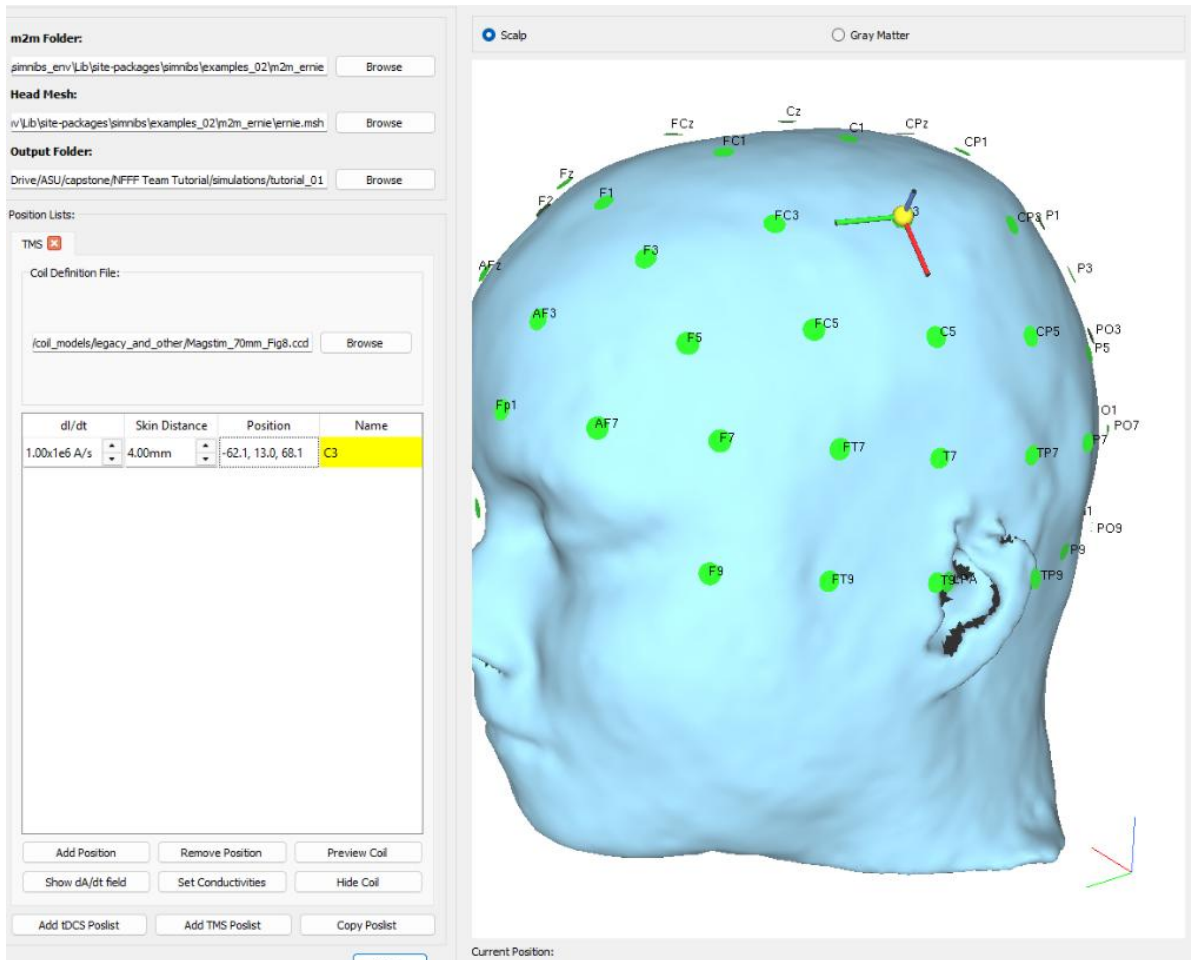
Select the C3 EEG coordinate. SimNIBS will populate the position and direction-reference coordinates. Use this standardized placement/orientation for the training run.

C3 is NOT the capstone target: C3 is only a convenient reproducible training location. NFFF research simulations will later use an appropriate left DLPFC target and project-specific positioning/orientation.

For fun, you can use the mouse to rotate and zoom the head model and inspect the stimulation position from several viewpoints.



NFFF SimNIBS Quick-Start Tutorial



6. Run the FEM Simulation

Click Run. A successful simulation should report:

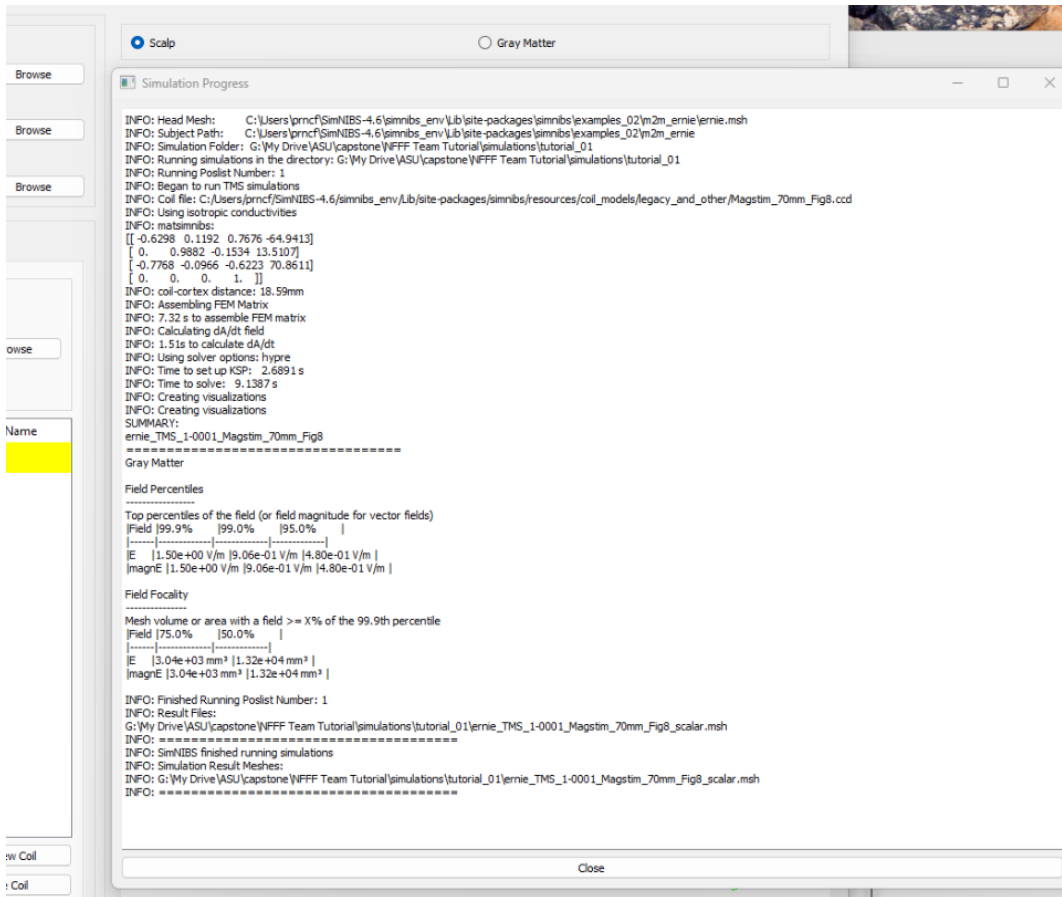
Finished running simulations

The important result file will have a name similar to:

ernie_TMS_1-0001_Magstim_70mm_Fig8_scalar.msh

This .msh file contains the finite-element mesh and calculated field data and may be several hundred MB.

NFFF SimNIBS Quick-Start Tutorial



7. Explore the Result in Gmsh

After a successful run, SimNIBS should open the result in Gmsh. The Post-processing panel includes useful layers:

magnE – Magnitude of the induced electric field, $|E| = \sqrt{E_x^2 + E_y^2 + E_z^2}$, in V/m. This is the main field used in the introductory analysis.

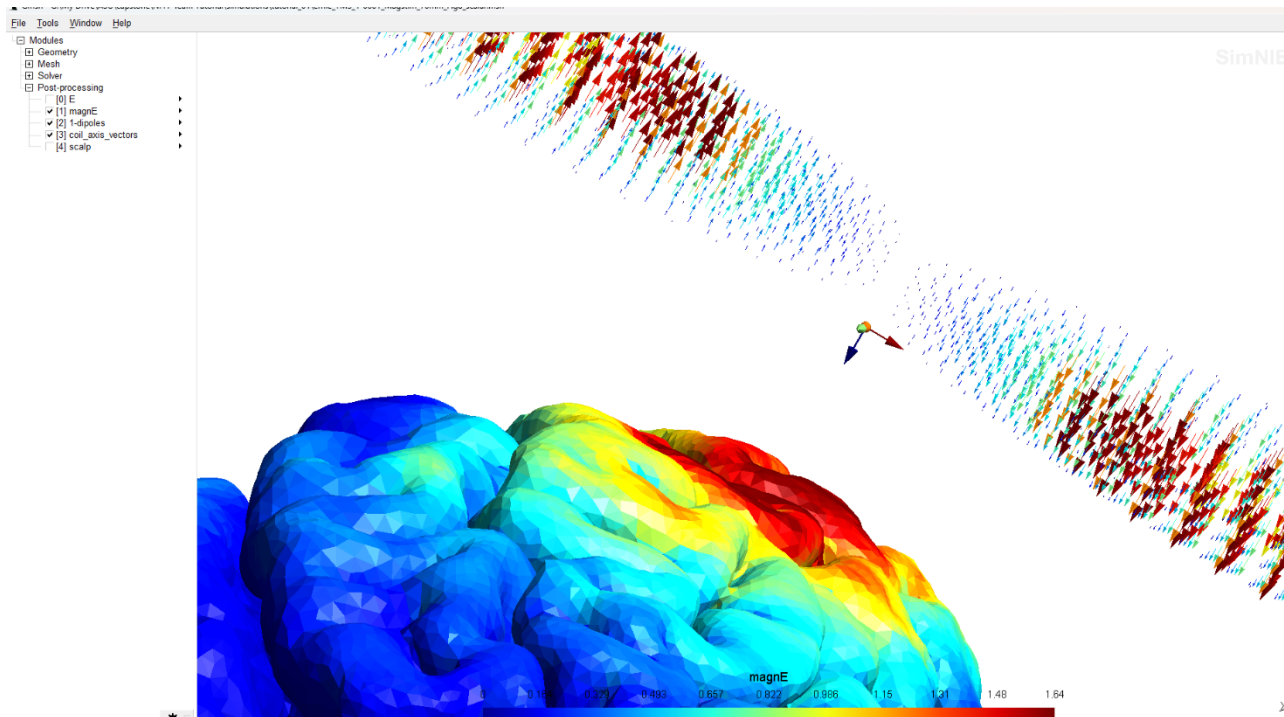
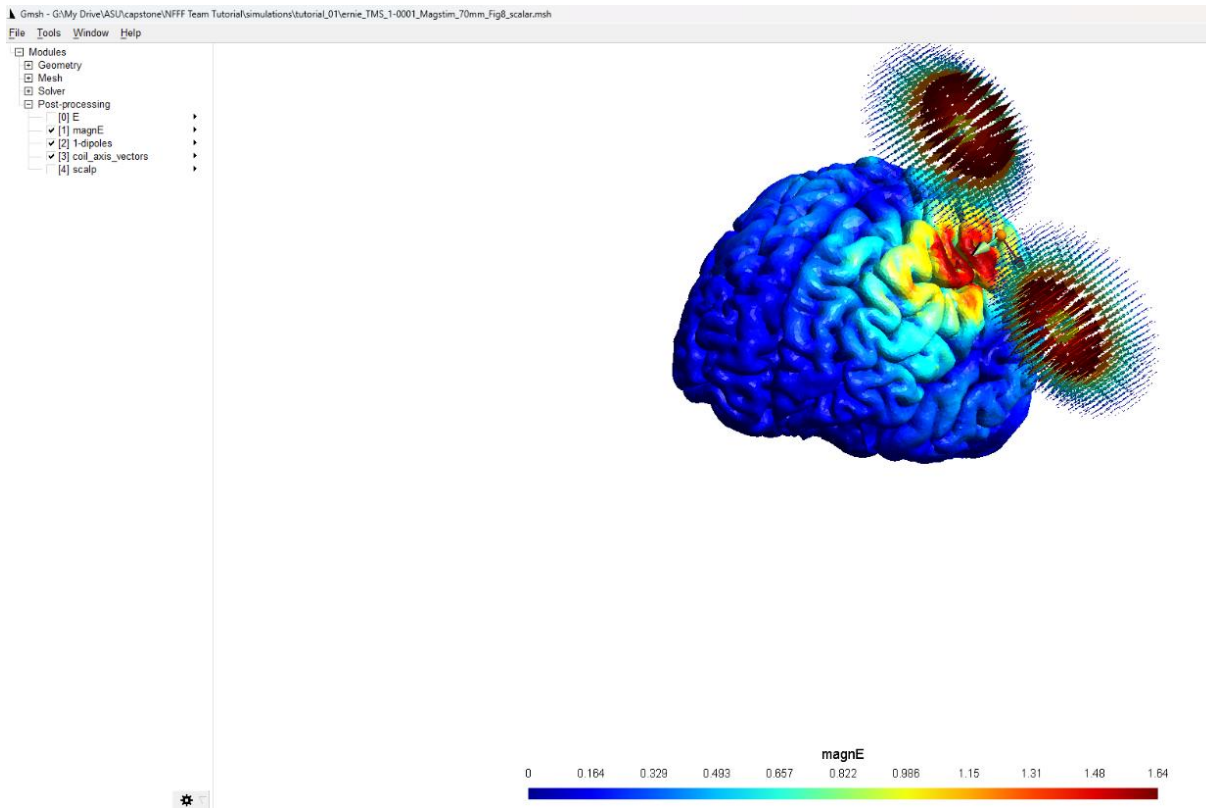
1-dipoles – The coil/source dipole representation used in the field calculation. Do not interpret these arrows as a direct plot of magnetic flux density B.

coil_axis_vectors – Vectors indicating coil position/orientation information.

Scalp – Scalp surface display that can be shown or hidden.

Use the mouse to rotate, zoom, and reposition the 3-D result. Turn layers on and off. Inspect the figure-8 source representation, cortex, and E-field distribution.

NFFF SimNIBS Quick-Start Tutorial



Recommended Python Environment for This Tutorial

For the NFFF SimNIBS data-extraction exercise, JupyterLab is the recommended and standardized Python environment.

NFFF SimNIBS Quick-Start Tutorial

JupyterLab is not required for Python in general. We recommend it here because SimNIBS installs its own Python environment, and Jupyter allows that environment to be registered and selected directly as a dedicated kernel named "Python - SimNIBS." This gives the entire team the same known-good setup and troubleshooting path.

Other environments such as Spyder, VS Code, or PyCharm can also be used by experienced team members, but they must be configured to use the SimNIBS Python interpreter/environment. We found that a normal Spyder installation did not readily access the SimNIBS environment, so configuring alternative IDEs is outside the scope of this introductory exercise.

For Tutorial 01, everyone should therefore use JupyterLab and the "Python - SimNIBS" kernel. After completing the standardized tutorial, experienced Python users are free to use another IDE if they configure it correctly.

If you do not already use Python/Jupyter regularly, follow this sequence exactly:

1. Install SimNIBS.
2. Activate the SimNIBS Python environment.
3. Register the "Python - SimNIBS" Jupyter kernel (one-time setup).
4. Launch JupyterLab from the NFFF Team Tutorial directory.
5. Open the supplied NFFF reference notebook.
6. Confirm that the notebook is using the "Python - SimNIBS" kernel.
7. Change the mesh path so it points to your own tutorial .msh file.
8. Run the notebook one cell at a time.

8. Register the SimNIBS Jupyter Kernel

A generic Jupyter Python kernel may not have access to the SimNIBS package. Register the SimNIBS environment as a dedicated kernel. This normally needs to be done only once per computer.

Open Windows Command Prompt and type the command below. The ".bat" at the end is part of the filename; do not type a separate word "bat":

```
C:\Users\<your_username>\simnibs_env\Scripts\activate.bat
```

Then register the environment:

```
python -m ipykernel install --user --name simnibs --display-name "Python - SimNIBS"
```

If already present: If "Python - SimNIBS" already appears in JupyterLab, do not repeat the registration.

9. Launch JupyterLab from the NFFF Working Directory

After activating simnibs_env, navigate to the tutorial directory. Example:

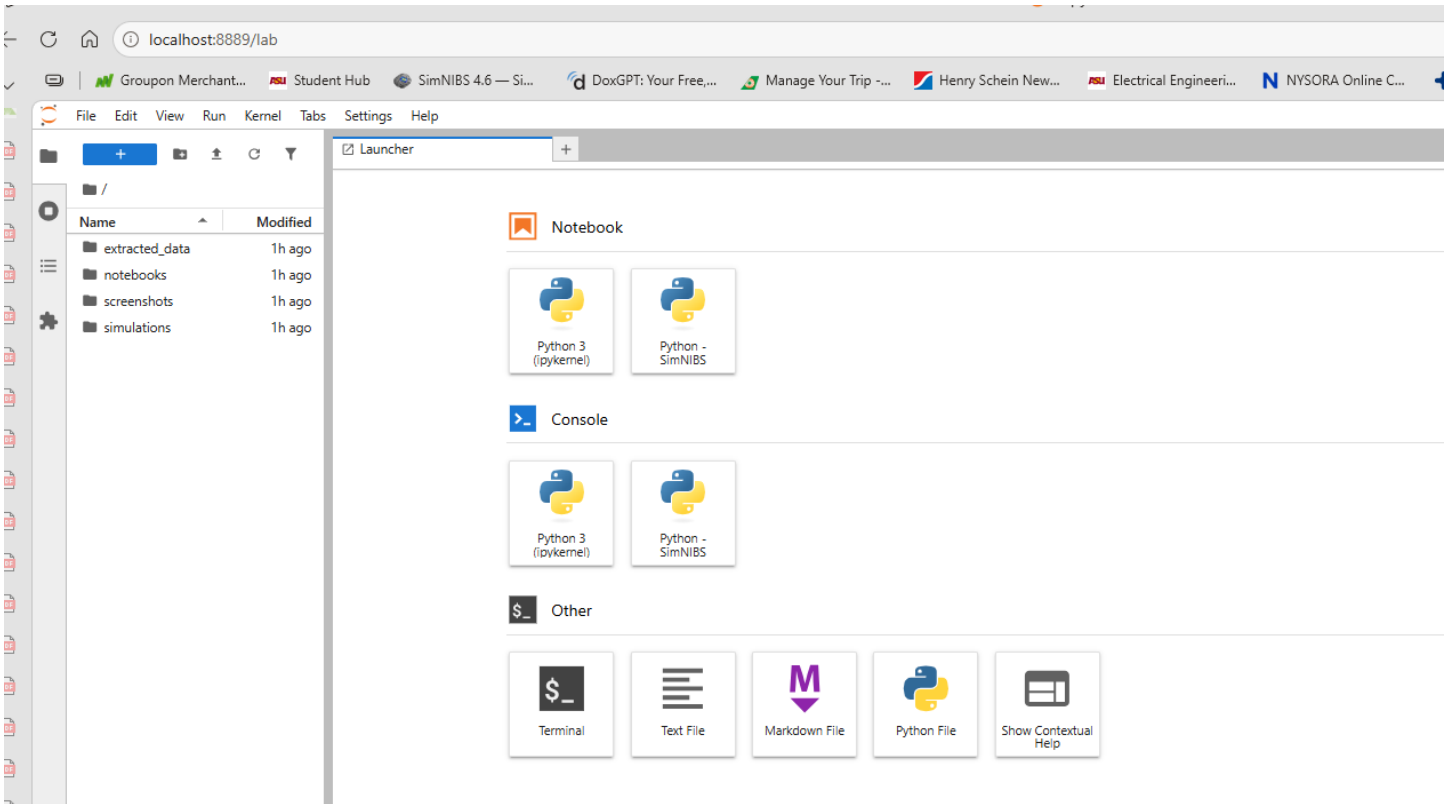
```
G:  
cd "G:\path\to\NFFF Team Tutorial"
```

Launch JupyterLab with:

```
python -m jupyterlab
```

Launching from the NFFF directory ensures that the Jupyter file browser can access the simulation, notebook, and extracted-data folders.

NFFF SimNIBS Quick-Start Tutorial



Team standard: For Tutorial 01, use JupyterLab even if you normally prefer another Python IDE. This keeps the SimNIBS environment consistent across the team and makes it much easier to help one another if a setup problem occurs.

10. Create the Data-Extraction Notebook

Open the notebooks folder and create a new notebook using "Python - SimNIBS", not the generic Python 3 kernel.

Suggested filename:

NFFF_SimNIBS_Data_Extraction_Tutorial.ipynb

11. Cell 1 - Imports and Mesh Path

```
# SimNIBS tools for reading and working with FEM mesh (.msh) files
from simnibs import mesh_io
```

```
# Numerical arrays and statistics
import numpy as np
```

```
# Plotting
import matplotlib.pyplot as plt
```

```
# Enter the path to YOUR tutorial FEM result
```

```
msh_path = (
    r"<your path>\NFFF Team Tutorial"
    r"\simulations\tutorial_01"
    r"\ernie_TMS_1-0001_Magstim_70mm_Fig8_scalar.msh"
)
```

NFFF SimNIBS Quick-Start Tutorial

```
print("Tutorial mesh:")
print(msh_path)
```

mesh_io is a module inside the SimNIBS Python package containing tools for reading and manipulating FEM mesh files. read_msh() reads a .msh file and creates a Python mesh object.

12. Cell 2 - Load the FEM Mesh

```
# Load the SimNIBS FEM result into Python
mesh = mesh_io.read_msh(msh_path)
```

```
print("Mesh loaded successfully.")
```

The mesh may take some time to load because the result contains millions of finite elements.

13. Cell 3 - Inspect the Stored Field Data

```
# Show the element-data fields stored in the mesh
for i, field in enumerate(mesh.elmdata):
    print(i, field.field_name)
```

For the validated tutorial run:

```
0 E
1 magnE
```

E is the vector electric field (Ex, Ey, Ez). magnE is the scalar magnitude $|E| = \sqrt{Ex^2 + Ey^2 + Ez^2}$.

14. Cell 4 - Extract magnE

```
# Extract electric-field magnitude
#
# mesh.elmdata[0] = E : vector electric field
# mesh.elmdata[1] = magnE : magnitude of electric field
#
# Units: V/m

magnE = mesh.elmdata[1].value

# magnE is now a NumPy array.
print("Number of magnE values =", magnE.size)

print("First 10 values:")
print(magnE[:10])
```

The validated tutorial mesh contained 5,576,894 magnE values.

15. Why We Do Not Analyze the Entire Mesh Indiscriminately

The full mesh contains multiple tissues and element categories. An unrestricted maximum can therefore be misleading if described as a cortical or gray-matter value. In the validated run, the unrestricted maximum was about 21.05 V/m, while the gray-matter peak was about 2.05 V/m.

Research rule: Always state the anatomical/tissue population or ROI over which a statistic is calculated.

NFFF SimNIBS Quick-Start Tutorial

16. Cell 5 - Inspect Tissue/Element Tags

```
# Inspect the tissue/element tags present in the mesh
tags = mesh.elm.tag1
unique_tags, counts = np.unique(tags, return_counts=True)

print("Tissue/element tags present in the mesh:")

for tag, count in zip(unique_tags, counts):
    print(f"Tag {tag}: {count:,} elements")
```

For this standard SimNIBS model, tissue tag 2 corresponds to gray matter. The validated tutorial run contained 1,337,567 tag-2 gray-matter elements.

17. Cell 6 - Create a Gray-Matter Mask

```
# Select gray matter
GRAY_MATTER = 2

# True = element is gray matter
# False = element is not gray matter
gm_mask = (mesh.elm.tag1 == GRAY_MATTER)

# Extract magnE only from gray-matter elements
magnE_gm = magnE[gm_mask]

print("Total magnE values =", magnE.size)
print("Gray-matter magnE values =", magnE_gm.size)
```

The Boolean mask does not modify the mesh. It creates a True/False selection that NumPy uses to keep only E-field values belonging to gray-matter elements.

18. Cell 7 - Gray-Matter E-Field Statistics

```
# Basic gray-matter electric-field statistics
print("----- Gray Matter Electric Field Statistics -----")

print("Peak E-field =", np.max(magnE_gm), "V/m")
print("Mean E-field =", np.mean(magnE_gm), "V/m")
print("95th percentile =", np.percentile(magnE_gm, 95), "V/m")
```

Validated metric	Approximate result
Peak E-field	2.0475 V/m
Mean E-field	0.1726 V/m
95th percentile	0.4982 V/m

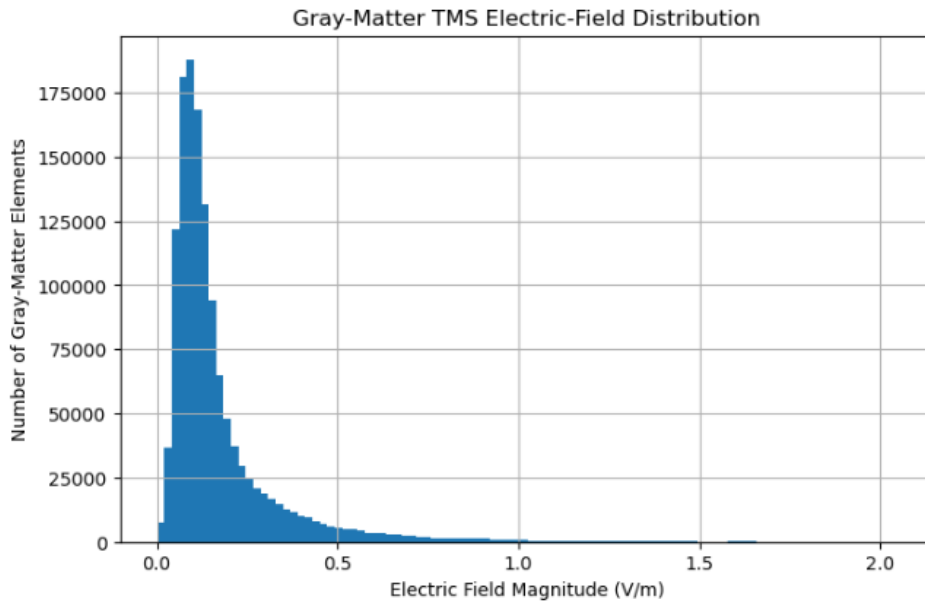
19. Cell 8 - Plot the Gray-Matter E-Field Distribution

```
# Histogram of gray-matter E-field magnitude
plt.figure(figsize=(8, 5))
plt.hist(magnE_gm, bins=100)

plt.xlabel("Electric Field Magnitude (V/m)")
plt.ylabel("Number of Gray-Matter Elements")
plt.title("Gray-Matter TMS Electric-Field Distribution")

plt.grid(True)
plt.show()
```

NFFF SimNIBS Quick-Start Tutorial



20. What You Just Accomplished

You completed the basic NFFF computational pipeline: standardized TMS coil placement -> MRI-derived head model -> SimNIBS FEM solution -> .msh result -> Gmsh visualization -> Python mesh object -> magnE extraction -> gray-matter selection -> quantitative statistics and plotting.

From here, individual roles diverge into coil modeling, anatomical targeting, data extraction/visualization, optimization, and clinical interpretation. Later analyses can add DLPFC ROIs, element-volume weighting, depth/focality metrics, comparison among coil configurations, basis-field extraction, and multi-coil optimization.

Congratulations

You have successfully run a TMS finite-element simulation, visualized the resulting electric field, extracted numerical E-field data from the FEM mesh, isolated gray matter, and analyzed the results in Python.

GitHub Note

A polished reference Jupyter notebook should be uploaded to the NFFF GitHub repository together with this tutorial. Team members do not need to retype every Python cell from scratch. Each person should first run the SimNIBS simulation locally, then open the supplied reference notebook in JupyterLab using the "Python - SimNIBS" kernel, point it to their own generated .msh file, and run the cells sequentially. Large generated .msh result files should not be committed to GitHub.